

University Number:

--	--	--	--	--	--	--	--	--	--

THE UNIVERSITY OF HONG KONG
SCHOOL OF COMPUTING AND DATA SCIENCE
DEPARTMENT OF COMPUTER SCIENCE

Final Examination

COMP7404D Computational Intelligence and Machine Learning

12 May 2025, 6:30 pm - 8:30 pm

Only approved calculators as announced by the Examinations Secretary can be used in this examination. It is the candidates' responsibility to ensure that their calculator operates satisfactorily, and candidates must record the name and type of the calculator used on the front page of the examination script.

Calculator Brand: _____ Calculator Model: _____

Answer all questions on this question paper in the space provided.
Write your University No. on every page and Seat No. below.

Seat No. _____

Question	Max Points	Your Points (examiner use only)
1	20	
2	30	
3	15	
4	15	
5	10	
6	10	
Total	100	

Question 1 [20 pts total]

Indicate whether each of the following statements is true or false by writing T or F in the provided box. Each correct answer earns 1 point, each incorrect answer deducts 1 point, and unanswered questions receive 0 points. The overall minimum for this question is 0 points. No explanation is necessary.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

1. The space complexity of Depth-First Search is generally lower than that of Breadth-First Search.
2. Local search algorithms, like hill climbing, maintain a frontier of nodes to explore multiple paths simultaneously.
3. Constraint Satisfaction Problems (CSPs) are typically solved using search algorithms that explore complete state assignments.
4. The Least Constraining Value (LCV) heuristic suggests choosing the value that rules out the most options for neighboring variables.
5. Minimax algorithm assumes that the opponent (MIN player) will always play optimally to minimize the MAX player's score.
6. Alpha-beta pruning can sometimes change the final minimax value computed at the root compared to a full Minimax search.
7. In an MDP, the transition function $T(s, a, s')$ specifies the reward obtained when moving from state s to s' via action a .
8. The optimal policy $\pi^*(s)$ for an MDP is always deterministic, meaning it specifies a single best action for each state.
9. Policy Iteration alternates between evaluating the current policy and improving it until convergence.
10. Solving an MDP (finding the optimal policy) requires the agent to interact with the environment to discover the transition and reward functions.
11. Temporal Difference (TD) learning updates value estimates only at the end of a complete episode.
12. Using function approximation (like linear features) in Q-learning allows generalization across states but may prevent convergence to the exact optimal Q-values.
13. The Perceptron algorithm uses a linear activation function to make its final classification prediction.
14. In Logistic Regression, L2 regularization helps prevent underfitting.
15. Batch Gradient Descent updates weights using the gradient computed over the entire training set.

University Number:

--	--	--	--	--	--	--	--	--	--

16. Random Forests typically have lower variance than individual decision trees trained on the same data.
17. The K-Nearest Neighbors (KNN) algorithm involves an explicit training phase to learn model parameters.
18. The Chinese Room Argument suggests that manipulating symbols according to rules is sufficient for genuine understanding.
19. Reinforcement Learning from Human Feedback (RLHF) uses human preferences to fine-tune language models towards desired behaviors like helpfulness and harmlessness.
20. Artificial General Intelligence (AGI) refers to AI systems designed for a single, specific task.

Question 2 [30 pts total]

For each of the following questions, select the single best answer from the options provided. Indicate your choice clearly by writing down your answer (a, b, c or d) in the provided box. Each correct answer earns 2 points. Incorrect answers and unanswered questions receive 0 points. No explanation is necessary.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

1. In the context of search problems, what does the "state space" represent?
a) The sequence of actions taken by the agent.
b) The set of all states reachable from the initial state.
c) The path cost from the start state to the goal state.
d) The data structure used to store nodes during search.
2. What is the primary difference between a Tree Search Algorithm (TSA) and a Graph Search Algorithm (GSA)?
a) TSA uses a stack, while GSA uses a queue.
b) GSA keeps track of explored states to avoid redundant paths, while TSA does not.
c) TSA always finds the optimal solution, while GSA does not.
d) GSA only works for graphs, while TSA only works for trees.
3. An admissible heuristic $h(n)$ used in A* search must satisfy which property, where $h^*(n)$ is the true cost to the nearest goal?
a) $h(n) > h^*(n)$
b) $h(n) = h^*(n)$
c) $0 \leq h(n) \leq h^*(n)$
d) $h(n)$ must be calculated using Manhattan distance.
4. In Constraint Satisfaction Problems (CSPs), what does "arc consistency" (like in AC-3) ensure?
a) That every variable has at least one value in its domain.
b) That for every value of a variable X, there is a compatible value for a neighboring variable Y connected by a constraint.
c) That a complete and consistent assignment exists.
d) That the constraint graph is a tree.
5. The Minimum Remaining Values (MRV) heuristic for variable ordering in CSPs suggests choosing which variable next?
a) The variable with the largest domain.
b) The variable involved in the most constraints.
c) The variable with the fewest legal values remaining.
d) The variable that was assigned last.
6. A Markov Decision Process (MDP) is characterized by the Markov property, which states that:
a) The next state's probability depends only on the current state and action, not the history.
b) All states are reachable from the start state.
c) The optimal policy is stationary.
d) The reward depends only on the current state.

7. Value Iteration is an algorithm used to compute:
- a) A fixed policy $\pi(s)$.

b) The transition function $T(s, a, s')$.

c) The reward function $R(s, a, s')$.

d) The optimal state values $V^*(s)$.
8. Q-Learning is considered an "off-policy" algorithm because:
- a) It learns the value of the optimal policy regardless of the policy being followed during exploration.

b) It requires a model of the environment.

c) It can only be used for deterministic policies.

d) It updates the policy directly without using Q-values.
9. In approximate Q-learning using linear function approximation, the Q-value $Q(s, a)$ is represented as:
- a) A weighted sum of features $f_i(s, a)$.

b) A table lookup.

c) A neural network output.

d) The average reward received for the pair (s, a) .
10. Feature scaling, such as standardization, is often important for gradient descent-based algorithms because:
- a) It ensures linear separability.

b) It prevents the algorithm from converging.

c) It converts categorical features into numerical ones.

d) It helps gradient descent converge more quickly and avoids issues caused by features having vastly different ranges.
11. Stochastic Gradient Descent (SGD) updates the model weights:
- a) After processing each individual training sample.

b) After processing a small batch of training samples.

c) After processing the entire training dataset.

d) Only when a misclassification occurs.
12. A major disadvantage of the KNN algorithm is:
- a) It requires extensive training time.

b) It is difficult to interpret.

c) Its prediction time can be slow for large datasets as it needs to compute distances to training samples.

d) It cannot handle multi-class problems.
13. The core idea behind Support Vector Machines (SVM) for classification is to find a hyperplane that:
- a) Passes through the most data points.

b) Minimizes the distance to the nearest data points.

c) Maximizes the margin between the different classes.

d) Connects the support vectors.
14. What technique allows SVMs to handle non-linearly separable data?
- a) Increasing the regularization parameter C.

b) Using gradient descent instead of quadratic programming.

c) Using kernels (like RBF) to map data to a higher-dimensional space.

d) Decreasing the number of support vectors.

15. Which of the following is listed as a major regulatory challenge for AI?
- a) Ensuring AI models can run on standard hardware.
 - b) Achieving transparency and explainability in complex AI models.
 - c) Developing faster training algorithms.
 - d) Translating AI outputs into different languages.

--	--	--	--	--	--	--	--	--	--

Question 3 [15 pts total]

The following Python function `bfs_graph_search` aims to implement the Breadth-First Search algorithm for a graph (GSA version), as discussed in the lecture slides.

Determine the missing lines of code indicated by `# --- BLANK X ---`. And fill them in the space provided on the next page. You should fill in exactly 3 lines to correctly implement the BFS graph search logic.

In this question, each node label is a single character, and a path is represented as a string of node labels (e.g., "SBG").

```
import collections
def bfs_graph_search(graph, start_node, goal_node):
    frontier = collections.deque([start_node])
    explored_set = set()
    while frontier:
        current_path = # --- BLANK 1 ---
        current_node = current_path[-1]
        if current_node == goal_node:
            return current_path
        if current_node not in explored_set:
            # --- BLANK 2 ---
            if current_node in graph:
                for neighbor in graph[current_node]:
                    new_path = current_path + neighbor
                    # --- BLANK 3 ---
    return None
example_graph = {
    'S': ['A', 'B'],
    'A': ['S', 'C'],
    'B': ['G'],
    'C': ['G'],
    'G': []
}
start = 'S'
goal = 'G'
path_found = bfs_graph_search(example_graph, start, goal)
if path_found:
    print(f"Path found: {path_found}")
else:
    print("Path not found.")
# Expected Output (assuming neighbors are processed in order):
# Path found: SBG
```

BLANK 1 [5 pts]

BLANK 2 [5 pts]

BLANK 3 [5 pts]

--	--	--	--	--	--	--	--	--	--

Question 4 [15 pts total]

The following Python code attempts to implement the Q-Learning algorithm for a simple 1D environment. The agent starts at state 0 and wants to reach the goal state 3. State 2 is a penalty state. Actions are 0 (move left) and 1 (move right).

```
import numpy as np
import random
NUM_STATES = 4          # States 0, 1, 2, 3
NUM_ACTIONS = 2         # 0: Left, 1: Right
GOAL_STATE = 3
PENALTY_STATE = 2
TERMINAL_STATES = [GOAL_STATE]
LEARNING_RATE = 0.1
DISCOUNT_FACTOR = 0.9
NUM_EPISODES = 500
EPSILON = 0.1
q_table = np.zeros((NUM_STATES, NUM_ACTIONS))
def get_next_state_reward(state, action):
    """ Simulates one step in the environment """
    if state == GOAL_STATE:
        return state, 0
    if state == 0: # Start state
        next_state = PENALTY_STATE if action == 0 else 1
    elif state == 1:
        next_state = 0 if action == 0 else GOAL_STATE
    elif state == 2: # Penalty state
        next_state = 1 if action == 0 else GOAL_STATE
    else: # Should not happen
        next_state = state
    if next_state == GOAL_STATE: reward = 10
    elif next_state == PENALTY_STATE: reward = -5
    else: reward = -1 # Cost of living/moving
    return next_state, reward

print("Starting Training...")
for episode in range(NUM_EPISODES):
    state = 0 # Start episode at state 0
    is_done = False
    while not is_done:
        if random.uniform(0, 1) < EPSILON:
            action = random.randint(0, NUM_ACTIONS - 1) # Explore
        else:
            action = np.argmax(q_table[state, :])
        next_state, reward = get_next_state_reward(state, action)
        max_next_q = np.max(q_table[next_state, :])
        td_target = reward + DISCOUNT_FACTOR * max_next_q
        td_error = td_target - q_table[state, action]
        q_table[state, action] = q_table[state, action] + LEARNING_RATE * td_error
        state = next_state
        if state in TERMINAL_STATES: is_done = True
    print("Training finished.")
    print("Q-Table:")
    print(np.round(q_table, 2))
```

There is a bug that prevents the algorithm from learning the correct Q-values. Identify the buggy line(s) of code and fix the bug in the space provided below by providing the corrected line(s) of code that should replace the buggy line(s).

--	--	--	--	--	--	--	--	--	--

Question 5 [10 pts total]

Consider the following Markov Decision Process (MDP):

- States (S): {S0, S1, S2}
- Actions (A): {Action A, Action B}
- Discount Factor (γ): 0.9
- Transition Probabilities $T(s, a, s')$ and Rewards $R(s, a, s')$:
 - From S0:
 - If Action A:
 - To S1 with probability 0.8, Reward = -1
 - To S0 with probability 0.2, Reward = -1
 - If Action B:
 - To S2 with probability 0.7, Reward = +10
 - To S1 with probability 0.3, Reward = -1
 - From S1:
 - If Action A:
 - To S0 with probability 0.6, Reward = -1
 - To S1 with probability 0.4, Reward = -1
 - If Action B:
 - To S2 with probability 0.9, Reward = +10
 - To S0 with probability 0.1, Reward = -1
 - From S2 (Terminal State):
 - Any action leads back to S2 with probability 1.0, Reward = 0.
- Current Value Estimates (V_k):
 - $V_k(S0) = 5$
 - $V_k(S1) = 8$
 - $V_k(S2) = 0$ (Value of terminal state is fixed at 0)
- Fixed Policy (π):
 - $\pi(S0) = \text{Action A}$
 - $\pi(S1) = \text{Action B}$

Calculate the updated value $V_{k+1}(S0)$. Show your work.

Question 6 [10 pts total]

Consider the following dataset of 2D points belonging to two classes, A and B:

Point ID	Feature 1 (X1)	Feature 2 (X2)	Class
P1	2	3	A
P2	4	5	A
P3	1	6	B
P4	5	2	B
P5	3	4	A
P6	6	4	B

You are given a new query point, **QP = (4, 3)**.

Using the K-Nearest Neighbors algorithm with **K = 3** and **Euclidean distance**, predict the class for the query point QP. In case of a tie among the nearest neighbors, select the class of the closest neighbor.

END OF PAPER

University Number:

--	--	--	--	--	--	--	--	--	--

This page was intentionally left blank